UAV-H3 Consensus: Lab Documentation

Exercise PARCHAM, Serial GAGAN, Jaisalmer sector

Environment: Ubuntu Linux VM, Python 3 (Pandas, NumPy, SciPy), virtual environment UAV3

1. Challenge

Six ground sensors recorded the arrival time (TOA) of 60 beacon bursts from a drone that stayed in one place. Two of the six sensors had been interfered with. Their data looked complete and normal, but it was not true. I had to find where the drone really was, to the nearest 10 m, and which two sensors were lying.

Flag format: KST{<north>_<east>_<letters>}

Files: README.md, sensors.csv, arrivals.csv

2. Looking at the data

I started with shell commands to see what the files contained before doing any analysis.

```
cat sensors.csv
head -8 arrivals.csv
wc -l arrivals.csv
```



Figure 1: Sensor positions, first rows of arrivals.csv and the line count

The six sensors are spread around the area, with A at the origin and the others between about 9 km and 18 km away from it. arrivals.csv has 361 lines. That is 360 data rows plus one header, which equals 60 bursts x 6 sensors, so every sensor reported every burst and there is no missing data.

The arrival times in burst 0 are all about 15.2758 s and differ only from the 5th decimal place. I worked out why: light travels about 300 m per microsecond, so a sensor 10 to 20 km away only adds around 30 to 70 microseconds. The 15.27 s is almost entirely the unknown time the drone chose to transmit. This means I cannot turn a TOA straight into a distance.

3. Measurement model and why I centred the data

For each sensor and each burst, the arrival time is:

```
TOA = t_tx + distance / c + noise
```

t_tx is the unknown transmit time of that burst. There are 60 of these, one per burst. If I subtract the average of the sensors in the same burst, t_tx cancels, because it is the same for every sensor in that burst:

```
TOA_i - TOA_mean = (d_i - d_mean)/c + (n_i - n_mean)
```

What remains depends only on distances (which depend on the drone position) and noise. So I never need to know the transmit time. This is the idea behind TDOA positioning.

The README gives timing noise of about 50 ns. At the speed of light that is 50 ns x 3x10^8 m/s = 15 m. So for a set of sensors that is honest, I expected the leftover error after the fit to be about 10 to 15 m, and for a set containing a bad sensor to be much larger.

4. Why I compared four-sensor subsets

Two of the six sensors are lying, and I do not know which. If I fit all six together, the false data is mixed with the true data and pulls the answer to a position that does not match anyone properly. If I pick four sensors, there is one choice of four that contains only honest sensors. There are 15 ways to choose 4 from 6. For each one I fit a position and measured the leftover error. The honest set should give the smallest error, close to the 15 m noise.

5. Fitting the position (fit.py)

For each subset the script did the following:

- Pivoted arrivals.csv into a 60 x 6 table, one row per burst and one column per sensor.
- For a trial position, calculated the distance to each sensor in the subset and the time light takes to travel it.
- Subtracted that travel time from each TOA, then subtracted the average of the subset in each burst (this removes t_tx).
- Multiplied the result by the speed of light to get the leftover error in metres. The raw values are around 1e-8 seconds, which is too small for the optimiser to handle, so I worked in metres.
- Used scipy least_squares to move the position until the error over all 60 bursts was as small as possible. The drone does not move, so one position has to fit all bursts.
- Tried five different starting positions per subset and kept the best, so the fit would not get stuck in a wrong answer.

The full script is in the appendix.

6. Results for all 15 subsets

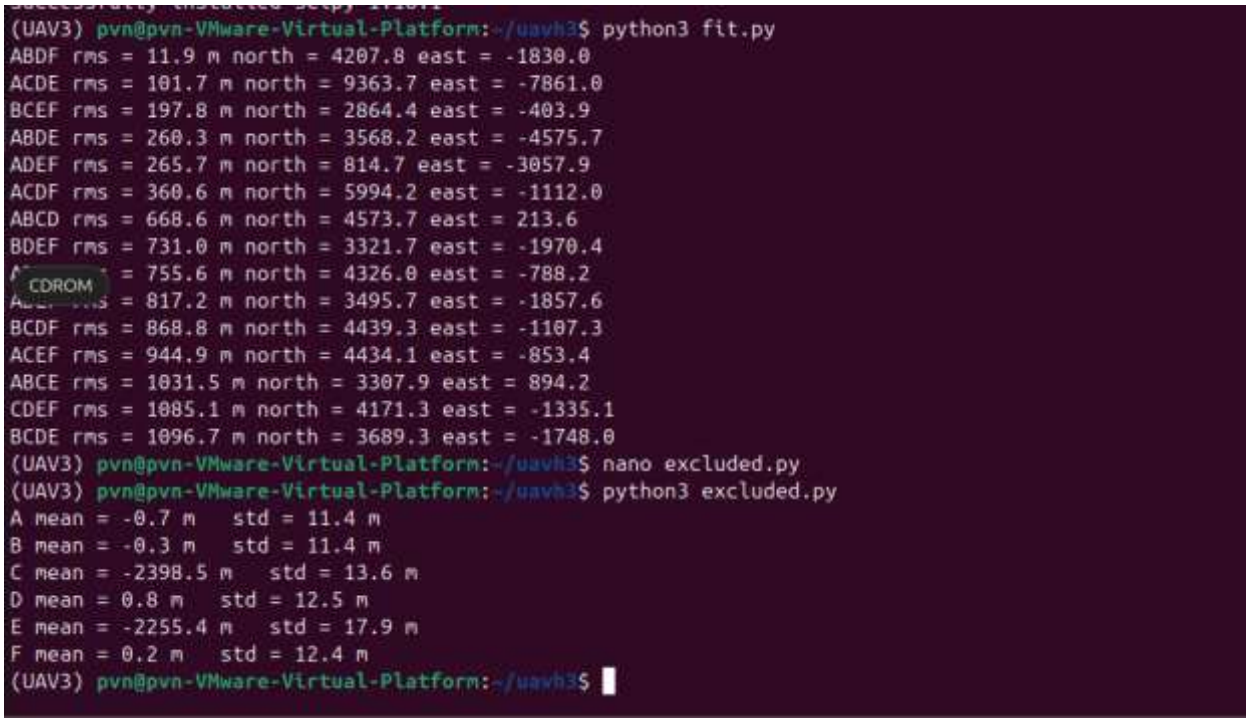


Figure 2: Output of fit.py and excluded.py

The top and the next few results were:

Subset	RMS error (m)	Fitted north (m)	Fitted east (m)
ABDF	11.9	4207.8	-1830.0
ACDE	101.7	9363.7	-7861.0
BCEF	197.8	2864.4	-403.9
ABDE	260.3	3568.2	-4575.7
ADEF	265.7	814.7	-3057.9

ABDF has an error of 11.9 m. The next best subset, ACDE, is 101.7 m, about 8.5 times worse. The rest go up to over 1,000 m. 11.9 m is just below the 15 m I expected from the noise, which makes sense because the fit absorbs a little of the noise. No other subset is anywhere near this, so ABDF is the honest set.

I noticed that ACDE and BCEF, the 2nd and 3rd best, both contain C and E together. Subsets with only one of them, such as ABCD (668.6 m), ABCF (755.6 m) or BDEF (731.0 m), did worse. I took this as a clue that C and E were shifted in a similar way, and I checked it in the next step.

7. Checking every sensor against the ABDF position (excluded.py)

Using the ABDF position, I estimated the transmit time of each burst from A, B, D and F only. Then, for all six sensors, I compared when the sensor should have heard the burst with when it actually reported it, in metres. An honest sensor should give about 0.

Sensor	Mean residual (m)	Std deviation (m)
A	-0.7	11.4
B	-0.3	11.4
C	-2398.5	13.6
D	0.8	12.5
E	-2255.4	17.9
F	0.2	12.4

A, B, D and F are all within 1 m of zero. C is about 2.4 km off and E is about 2.3 km off. Their standard deviations (13.6 m and 17.9 m) are about the same as the honest sensors (11 to 12 m). So the two bad sensors are not noisy. Each one has a fixed shift on top of normal-looking noise. This explains why the README says nothing looks wrong when each is examined alone. A constant shift looks like an unknown transmit time, so only the agreement of the four other sensors exposes it.

8. Result

Item	Value
Trusted sensors	A, B, D, F
Compromised sensors	C, E
Fitted position	North 4207.8 m, East -1830.0 m
Rounded to 10 m	North 4210, East -1830
Flag	KST{4210_-1830_CE}

4207.8 rounds to 4210. -1830.0 is already a multiple of 10, and I kept the minus sign because the drone is west of the origin. The letters are in alphabetical order with nothing between them. This agrees with the independent check I was given.

9. Limits and what I would do next

- The result depends on assuming the drone was stationary and that only two sensors were lying. If three were compromised, no four-sensor subset would be clean.
- The margin is large (11.9 m against 101.7 m), so I am confident, but I would still check the fit with a different optimiser or start points.
- I did not use the spoofed offsets to learn anything else about the interference, such as whether the two offsets are related.

10. If asked to explain it

Each sensor reports the transmit time plus the travel time. The transmit time is unknown but the same for every sensor, so subtracting the average in each burst removes it. Two sensors are lying, so I tried all 15 groups of four and fitted one stationary position to each. Only one group, ABDF, fits at the noise level of about 12 m. Using that position, C and E are off by more than 2 km, so they are the compromised sensors.

Appendix: scripts

fit.py

```
import itertools
import numpy as np
import pandas as pd
from scipy.optimize import least_squares

C = 299792458.0
S = pd.read_csv("sensors.csv").set_index("sensor")
A = pd.read_csv("arrivals.csv")
T = A.pivot(index="burst_id", columns="sensor", values="toa_s")

def residuals(pos, sub):
    d = np.array([np.hypot(S.north_m[s]-pos[0], S.east_m[s]-pos[1]) for s in sub]) /
C
    x = T[list(sub)].values - d
    x = x - x.mean(axis=1, keepdims=True)
    return (x * C).ravel()

results = []
for sub in itertools.combinations("ABCDEF", 4):
    best = None
    for guess in [(0,0),(3000,-2000),(5000,1000),(-3000,3000),(2000,4000)]:
        r = least_squares(residuals, guess, args=(sub,))
        if best is None or r.cost < best.cost:
            best = r
    rms = np.sqrt(np.mean(best.fun ** 2))
    results.append((rms, "".join(sub), best.x))

for rms, sub, pos in sorted(results):
    print(sub, "rms = %.1f m" % rms, "north = %.1f east = %.1f" % tuple(pos))
```

excluded.py

```
    import numpy as np
    import pandas as pd

    C = 299792458.0
    S = pd.read_csv("sensors.csv").set_index("sensor")
    A = pd.read_csv("arrivals.csv")
    T = A.pivot(index="burst_id", columns="sensor", values="toa_s")

    trusted = "ABDF"
    pos = (4207.77, -1829.95)

    def delay(s):
        return np.hypot(S.north_m[s]-pos[0], S.east_m[s]-pos[1]) / C

    tx = np.mean([T[s].values - delay(s) for s in trusted], axis=0)

    for s in "ABCDEF":
        r = (T[s].values - delay(s) - tx) * C
        print(s, "mean = %.1f m   std = %.1f m" % (r.mean(), r.std()))
```